Integrantes:

Vanessa Judith Garza Fernández 2145305

Andrick Fabian Moreno Torres 2147624

Edmundo Santos Ramírez 2148132

Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Actividad 4
Fecha: 27/10/2026	

Bitácora de escenarios E01 a E15

Cada escenario se ejecutó con el script ejecutar_escenarios.py, recargando los datos (load_fixture) antes de cada caso.

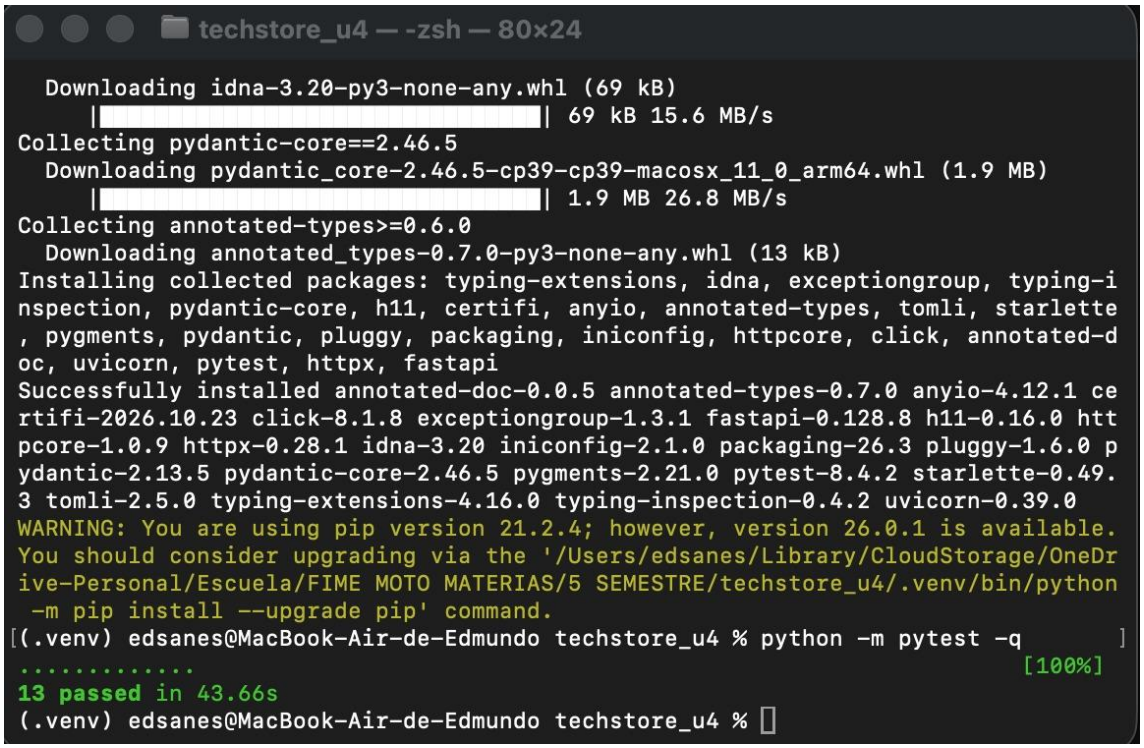
Caso	Estado inicial	Operación	Resultado real	Explicación	Evento emitido
E01 (O001)	DRAFT	Evaluar descuento	Descuento 10%	Cliente VIP, total 2100 > 500, 4 unidades (>= 4) y sin promoción: se cumple la regla del 10%.	Ninguno (solo se evalúa)
E02 (O002)	DRAFT	Evaluar descuento	Descuento 0%	No es VIP y el total (1050) no supera 5000: no aplica ninguna regla.	Ninguno (solo se evalúa)
E03 (O003)	DRAFT	Confirmar	Rechazado: InvalidOrderData	El cliente está inactivo, así que no se puede confirmar. Sigue en DRAFT.	Sin evento (falló)
E04 (O005)	DRAFT	Evaluar descuento	Descuento 0%	Tiene promoción activa, lo que bloquea tanto el 10% como el 5%.	Ninguno (solo se evalúa)
E05 (O006)	DRAFT	Evaluar descuento	Descuento 5%	No cumple el 10% (solo 1 unidad), pero el total 18000 > 5000 y sin promoción: aplica el 5%.	Ninguno (solo se evalúa)
E06 (O007)	DRAFT	Confirmar	Rechazado: InvalidOrderData	El pedido no tiene artículos. Sigue en DRAFT.	Sin evento (falló)
E07 (O008)	DRAFT	Confirmar	Éxito: pasa a CONFIRMED	20 unidades exactas están permitidas (el límite es 20).	OrderConfirmed
E08 (O009)	DRAFT	Agregar SKU003 x2	Rechazado: OrderItemLimitExceeded	19 + 2 = 21 unidades, excede el límite. No se modificó items: sigue en 19 unidades.	Sin evento (falló)
E09 (O010)	DRAFT	Evaluar descuento	Descuento 0%	El total es exactamente 500 y el umbral es estricto (> 500), por eso no aplica.	Ninguno (solo se evalúa)
E10 (O011)	DRAFT	Evaluar descuento	Descuento 0%	Solo tiene 3 unidades y la regla pide al menos 4.	Ninguno (solo se evalúa)

Caso	Estado inicial	Operación	Resultado real	Explicación	Evento emitido
E11 (O012)	DRAFT	Confirmar y enviar sin guía	Confirma bien; envío rechazado: InvalidOrderData	La confirmación es válida, pero enviar requiere número de guía. Queda en CONFIRMED.	OrderConfirmed (el envío no emitió evento)
E12 (O013)	DRAFT	Confirmar dos veces	2a confirmación rechazada: InvalidOrderTransition	Solo se confirma desde DRAFT. Sigue en CONFIRMED y no se duplica el evento.	OrderConfirmed (uno solo)
E13 (O014)	DRAFT	Confirmar, enviar y cancelar	Cancelar rechazado: InvalidOrderTransition	Solo se cancela desde CONFIRMED; un pedido enviado no se puede cancelar. Queda en SHIPPED.	OrderConfirmed, OrderShipped
E14 (O015)	DRAFT	Confirmar, enviar y devolver	Éxito: termina en RETURNED	Flujo válido DRAFT, CONFIRMED, SHIPPED y RETURNED.	OrderConfirmed, OrderShipped, OrderReturned
E15 (O016)	DRAFT	Confirmar, enviar y entregar	Éxito: termina en DELIVERED	Flujo válido DRAFT, CONFIRMED, SHIPPED y DELIVERED.	OrderConfirmed, OrderShipped, OrderDelivered

Pruebas con pytest

Se escribieron 9 pruebas de dominio (tests/test_domain.py) y 4 de la API (tests/test_api.py): Money negativo, 21 unidades, cliente inactivo, pedido vacío, descuento VIP, promoción activa, envío sin guía, confirmación doble, cancelación desde SHIPPED y los códigos HTTP 404, 200, 409 y 422.

Captura de python -m pytest -q



Capturas de Swagger

Documentación en http://127.0.0.1:8000/docs.

Operación exitosa: POST /orders/O001/confirm (200)



Operación rechazada: POST /orders/O013/confirm por segunda vez (409)



Cálculo manual

Se desarrollo paso a paso los cálculos y la lógica de las reglas de descuento.

1. Fórmulas base

Subtotal de un renglón = cantidad x precio unitario

Total del pedido (T) = es la suma de los subtotales de todos sus renglones

Unidades del pedido (U) = es la suma de las cantidades de todos sus renglones

Las unidades se obtienen sumando cantidades, no se cuentan renglones.

2. Pedido O001 (cliente C001)

Renglón 1: SKU001, cantidad 2, precio 600.00 → 2 × 600.00 = 1200.00

Renglón 2: SKU002, cantidad 2, precio 450.00 → 2 × 450.00 = 900.00

T = 1200.00 + 900.00 = 2100.00 MXN

U = 2 + 2 = 4

Resultado: 0001 → cliente C001 → 4 unidades → 2100.00 MXN.

3. Pedido O006 (cliente C006)

Datos de la guía: U = 1 y T = 18000.00. Con una sola unidad hay un único renglón con cantidad 1:

T = 1 x precio = 18000.00 → precio = 18000.00 / 1 = 18000.00 MXN

Resultado: O006 → cliente C006 → 1 unidad → 18000.00 MXN.

4. Pedido O010 (cliente C010)

Datos de la guía: U = 2 y T = 500.00. Hay dos posibilidades:

(a) Un renglón con cantidad 2: 2 × precio = 500.00 → precio = 500.00 / 2 = 250.00

(b) Dos renglones con cantidad 1: precio A + precio B = 500.00

Resultado: O010 → cliente C010 → 2 unidades → 500.00 MXN, lo importante para las reglas es que T vale exactamente 500.00.

5. Tabla de la práctica

Pedido	Cliente	Unidades	Total manual
O001	C001	4	2100.00 MXN
O006	C006	1	18000.00 MXN
O010	C010	2	500.00 MXN

6. Reglas de descuento

Cada condición vale 1 si se cumple y 0 si no:

- **M** = el total es mayor que 500 (T > 500, estricto).
- **V** = el cliente es VIP.
- **N** = las unidades son al menos 4 (U ≥ 4).
- **P** = el pedido tiene promoción activa.
- **G** = el total es mayor que 5000 (T > 5000, estricto).

Regla del 10 %: $D10 = M \land V \land N \land \neg P$
Regla del 5 %: $D5 = G \land \neg P$
Combinadas: $D = D10 \lor D5$

Simplificación, con ¬P como factor común (propiedad distributiva)

$D = (M \land V \land N \land \neg P) \lor (G \land \neg P)$
$D = \neg P \land [(M \land V \land N) \lor G]$

en conckusion si hay promoción activa (P = 1), ningún descuento se aplica.

Tasa de descuento con prioridad al 10 %:

Si D10 = 1 → descuento de 10 %
Si D10 = 0 y D5 = 1 → descuento de 5 %
En cualquier otro caso → descuento de 0 %

6.1 Evaluación de O001 (T = 2100.00, U = 4, cliente VIP, sin promoción)

M: 2100 > 500 → 1 V: VIP → 1 N: 4 ≥ 4 → 1 ¬P: sin promoción → 1
$D10 = 1 \land 1 \land 1 \land 1 = 1$

Resultado: descuento de 10 %.

6.2 Evaluación de O006 (T = 18000.00, U = 1, sin promoción)

N: 1 ≥ 4 → 0, por lo que D10 = 0 (basta que una condición falle).
G: 18000 > 5000 → 1 ¬P: sin promoción → 1
$D5 = 1 \land 1 = 1$

Resultado: descuento de 5 %.

6.3 Evaluación de O010 (T = 500.00, U = 2)

M: 500 > 500 → 0 (500 no es mayor que 500), por lo que D10 = 0.
G: 500 > 5000 → 0, por lo que D5 = 0.

Resultado: descuento de 0 %. También N falla (2 < 4), pero con M = 0 ya basta.

6.4 Resumen de los escenarios de descuento

Caso	Pedido	Condición determinante	Tasa
E01	O001	M, V, N y sin promoción se cumplen	10 %

Caso	Pedido	Condición determinante	Tasa
E02	O002	No es VIP y 1050 no supera 5000	0 %
E04	O005	Tiene promoción activa (¬P = 0)	0 %
E05	O006	Solo 1 unidad (N = 0), pero 18000 > 5000	5 %
E09	O010	500 no supera 500	0 %
E10	O011	Solo 3 unidades (N = 0)	0 %

7. Importe del descuento

La práctica solo exige calcular la tasa. Si se aplicara al total:

O001: 2100.00 × 0.10 = 210.00 → 2100.00 − 210.00 = 1890.00 MXN
O006: 18000.00 × 0.05 = 900.00 → 18000.00 − 900.00 = 17100.00 MXN

8. Límite de 20 unidades

Se rechaza el artículo cuando: unidades actuales + cantidad nueva > 20.

- **O009:** 19 + 2 = 21 y 21 > 20, por lo que se rechaza con OrderItemLimitExceeded. La validación va antes de agregar el renglón, así que las unidades siguen en 19.
- **O008:** llega a 20 unidades y 20 > 20 es falso, por lo que es válido y confirma.

9. Por qué Decimal y no float

Con float: 0.1 + 0.2 = 0.30000000000000004
Con Decimal: Decimal("0.1") + Decimal("0.2") = Decimal("0.3")

Con float una comparación como T > 500 puede fallar por error de representación binaria. Decimal construido desde texto es exacto; por eso se escribe Decimal("600.00") y nunca Decimal(600.00).

10. Conteo de transiciones de la máquina de estados

Hay 6 estados (DRAFT, CONFIRMED, SHIPPED, DELIVERED, CANCELLED, RETURNED) y 5 operaciones (confirm, ship, cancel, deliver, return_order).

Pares (estado, operación) = 6 × 5 = 30

Solo 5 transiciones son válidas: DRAFT → CONFIRMED, CONFIRMED → SHIPPED, CONFIRMED → CANCELLED, SHIPPED → DELIVERED y SHIPPED → RETURNED.

Pares que deben lanzar InvalidOrderTransition = 30 − 5 = 25

Verificación con O014 (E13): confirm() → CONFIRMED con 1 evento; ship("GUIA1") → SHIPPED con 2 eventos; cancel("motivo") → rechazado, sigue en SHIPPED con 2 eventos.

README

Propósito

implementar el núcleo de negocio de una tienda en línea con entidades y Value Objects que protegen sus invariantes, reglas de descuento con el patrón Specification (AND/OR/NOT), una máquina de estados con Domain Events y una API REST con FastAPI.

Problema que resuelve

Evita que se confirmen pedidos inválidos (vacíos, de clientes inactivos o con más de 20 unidades), aplica descuentos de forma consistente y rechaza transiciones de estado prohibidas, de modo que las reglas viven en el dominio y no en la interfaz. Los datos se cargan desde CSV y se manejan en memoria; al cerrar el programa, los cambios se pierden.

Reglas principales

- Límite de 20 unidades por pedido (suma de cantidades, no de renglones).
- Descuento de 10 %: total > 500, cliente VIP, al menos 4 unidades y sin promoción activa.
- Descuento de 5 %: total > 5000 y sin promoción activa.
- Si se cumplen ambos, se aplica el 10 %.
- Estados: DRAFT → CONFIRMED → SHIPPED → DELIVERED; CONFIRMED → CANCELLED; SHIPPED → RETURNED.
- Una operación inválida no cambia el estado ni agrega eventos.

Requisitos

- Python 3.11 o superior.
- Dependencias: fastapi, uvicorn, pytest y httpx.
- No requiere base de datos ni permisos de administrador.

Instrucciones de ejecución / uso

```
# 1. Entorno virtual
python -m venv .venv
# Windows PowerShell: .venv\Scripts\Activate.ps1
# Windows CMD:        .venv\Scripts\activate.bat
# macOS / Linux:      source .venv/bin/activate

# 2. Dependencias
python -m pip install fastapi uvicorn pytest httpx

# 3. Verificar la carga de datos (16 pedidos en DRAFT)
python cargar_csv.py

# 4. Ejecutar los 15 escenarios
python ejecutar_escenarios.py

# 5. Pruebas automáticas
python -m pytest -q

# 6. Levantar la API
uvicorn app.api:app --reload
```

Documentación: http://127.0.0.1:8000/docs

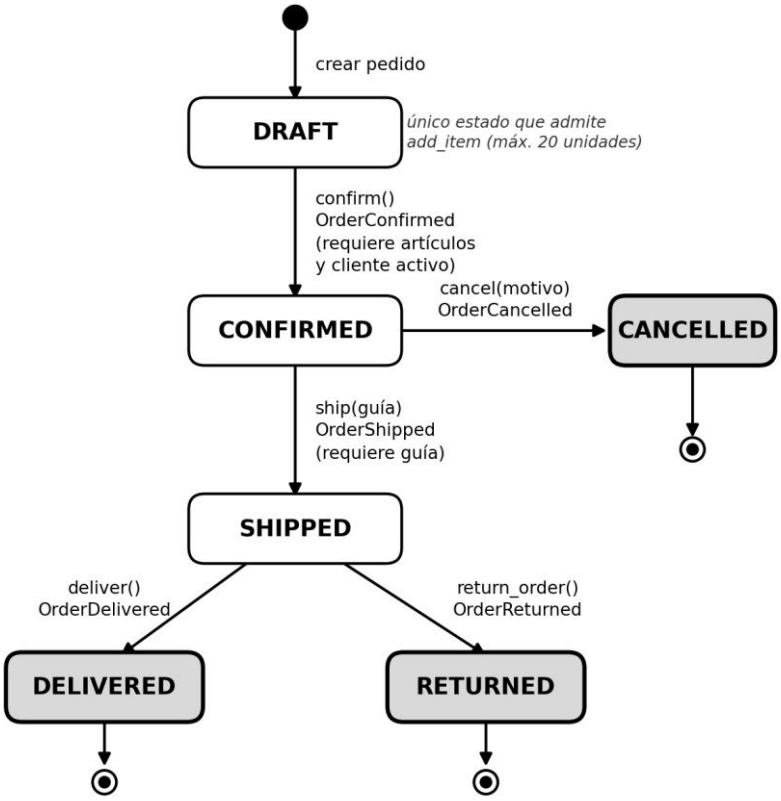
Endpoints

Método	Ruta	Descripción	Respuestas
GET	/orders/{id}	Consulta un pedido	200, 404
POST	/orders/{id}/confirm	Confirma el pedido	200, 404, 409, 422
POST	/orders/{id}/ship	Envía (body: tracking_number)	200, 404, 409, 422
POST	/orders/{id}/cancel	Cancela (body: reason)	200, 404, 409, 422

Estructura del proyecto

```
techstore-u4/
  datos/            # CSV de clientes, productos, pedidos, detalle y escenarios
  cargar_csv.py
  ejecutar_escenarios.py
  app/
    __init__.py
    domain.py
    specifications.py
    api.py
```

Diagrama de estados



Explicación del diagrama

- **Estado inicial:** DRAFT, aquí se agregan artículos (máximo 20 unidades en total) y es donde comienzan los 16 pedidos del CSV.
- **Estados intermedios:** CONFIRMED y SHIPPED.
- **Estados finales:** DELIVERED, CANCELLED y RETURNED. Desde ellos no sale ninguna transición.

Transición	Condiciones para dispararla	Evento	Si no se cumple
DRAFT → CONFIRMED	confirm(): estado DRAFT, al menos un artículo y cliente activo	OrderConfirmed	409 o 422
CONFIRMED → SHIPPED	ship(guía): estado CONFIRMED y número de guía no vacío	OrderShipped	409 o 422
CONFIRMED → CANCELLED	cancel(motivo): estado CONFIRMED y motivo no vacío	OrderCancelled	409 o 422
SHIPPED → DELIVERED	deliver(): estado SHIPPED	OrderDelivered	409
SHIPPED → RETURNED	return_order(): estado SHIPPED	OrderReturned	409

Domain Events

Un domain event es un objeto inmutable con el nombre del hecho, el id del pedido y la fecha y hora en UTC, su nombre va en pasado porque comunica algo que ya ocurrió cada método de Order valida primero, cambia el estado después y registra el evento al final, por lo que una operación rechazada no cambia el estado ni agrega eventos. Cualquier salto que no aparezca en el diagrama, como cancelar desde SHIPPED, está prohibido y lanza InvalidOrderTransition.

Reflexión

La Actividad 4 tuvo como propósito crear un núcleo de negocio ejecutable: entidades y Value Objects con invariantes, reglas de descuento mediante el patrón specification, una máquina de estados con Domain Events y una API REST que funciona solo como frontera de entrada, el caso de TechStore permitió comprobar que las políticas comerciales pueden vivir en el dominio y probarse sin servidor web ni base de datos.

Como equipo enfrentamos tres retos principales. El primero fue distinguir entre contar renglones y sumar unidades: el límite de 20 corresponde a la suma de cantidades, por lo que add_item debe validar antes de modificar la lista, el segundo fueron los umbrales estrictos: el pedido O010, con total exacto de 500.00 MXN, obligó a usar «>» en el monto y «>=» en las unidades, además de construir decimal desde texto para evitar errores de redondeo del float, el tercero fue garantizar que una operación inválida no dejara rastro; lo resolvimos siguiendo siempre el orden validar, cambiar el estado y registrar el evento.

Concluimos que centralizar las reglas en el dominio reduce errores y facilita las pruebas, y que permite sustituir la interfaz sin reescribir la lógica. También aprendimos que una prueba debe demostrar tanto el caso válido como el rechazo correcto.

Preguntas del reporte

1. ¿Por qué se usa Decimal en lugar de float para precios?

Porque float guarda los números en binario y muchos decimales no se pueden representar con exactitud en python 0.1 + 0.2 da 0.30000000000000004, en dinero esos errores se acumulan y pueden hacer fallar comparaciones como total > 500, que en esta práctica deciden si se aplica un descuento, por otro lado ecimal trabaja en base decimal y es exacto siempre que se construya desde texto, por eso se escribe decimal("600.00") y nunca decimal(600.00) que arrastraría el error del float además, money valida que el monto sea finito y no negativo desde su creación.

2. ¿Cuál es la diferencia entre contar renglones y sumar unidades?

Un renglón es una línea del detalle del pedido (un sku con su cantidad) mientras que las unidades son la suma de las cantidades de todos los renglones por ejemplo 0001 tiene 2 renglones 2 × SKU001 y 2 × SKU002 pero 4 unidades, el límite de 20 y la regla de descuento de al menos 4 unidades se miden en unidades si se contaran renglones, un pedido con pocas líneas de cantidades grandes superaría el límite sin que el sistema lo detectara por eso 0009 con 19 unidades rechaza agregar 2 más (19 + 2 = 21), y 0008 con 20 unidades exactas sí se confirma.

3. ¿Qué ventaja tiene encapsular una regla en Specification?

Cada condición del negocio queda como un objeto con nombre que se puede reutilizar y probar por separado, las reglas se combinan con los operadores AND, OR y NOT sin repetir expresiones booleanas largas en distintos lugares, por ejemplo la del 10 %: monto mínimo AND vip AND unidades mínimas AND NOTpromoción activa, Si cambia un umbral se modifica en un solo sitio y como la Specification solo evalúa y responde verdadero o falso nunca modifica el pedido.

4. ¿Por qué los eventos se registran solo después de una transición válida?

Porque un Domain Event comunica un hecho que ya sucedió si se registrara antes de validar, una operación rechazada anunciaría algo que nunca ocurrió,

5. ¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?

Casi todo porque las reglas no viven en la API se reutilizarían domain, specifications, cargar_csv y las pruebas de dominio, solo habría que reemplazar api que contiene los controladores, los modelos de petición y respuesta y la traducción de excepciones a códigos HTTP, una interfaz de consola, una aplicación de escritorio o un proceso por lotes llamarían los mismos métodos confirm(), ship(), cancel(), deliver() y return_order(); ejecutar_escenarios ya demuestra esto, pues ejecuta los 15 escenarios sin servidor web.

LINK DEL REPOSITORIO

https://github.com/afmt312-wq/Actividad4-COMLOGN.git